

FIT2109 Computer Science Workshop

Week 8: Debugging — Systematic Bug Hunting

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

Debugging is most of the job

"Everyone knows that debugging is twice as hard as writing a program in the first place. So if you are as clever as you can be when you write it, how will you ever debug it?"

— Brian Kernighan

- Studies consistently show developers spend 35–50% of their time debugging.
- Most bugs are not mysterious — they have a cause that can be found systematically.
- The difference between a good and a struggling developer is often *method*, not knowledge.

By the end of this workshop, you should be able to:

- Apply a structured debugging workflow: **reproduce** → **isolate** → **inspect** → **fix** → **verify**.
- Read error messages and tracebacks and identify the true root cause.
- Interpret log output using knowledge of log levels.
- Use static analysis tools to catch bugs before running code.
- Set breakpoints and inspect program state with GDB/LLDB and the VS Code debugger.
- Construct a minimal reproducible example and use Perses to automate reduction.

When you encounter a bug, what is the first thing you do?

Concept 1: Debugging is disciplined hypothesis testing

Apply the scientific method

- 1 **Observe:** what exactly goes wrong?
- 2 **Hypothesise:** what could cause this?
- 3 **Experiment:** make one change, re-run.
- 4 **Conclude:** was the hypothesis right?
- 5 Repeat until fixed.

Common traps

- Changing multiple things at once.
- Assuming the bug is in someone else's code.
- Fixing symptoms, not causes.

The workflow

Reproduce



Isolate



Inspect



Fix



Verify

Concept 2: Error messages tell you where to start

Read a Python traceback from the bottom up

```
Traceback (most recent call last):  
  File "app.py", line 18, in <module>  
    result = process(data)  
  File "app.py", line 9, in process  
    return data[index]  
IndexError: list index out of range
```

Reading order

- 1 **Bottom line:** the error type and message.
- 2 **Line above:** the exact file and line.
- 3 **Stack:** how we got there.

Common error types

SyntaxError Python can't parse the file.

NameError Variable not defined.

TypeError Wrong type for operation.

IndexError List access out of range.

KeyError Dict key missing.

AttributeError Object has no such attribute.

Concept 3: Log levels separate noise from emergencies

Five standard levels (low to high)

Level	Use for	Default?
DEBUG	Fine-grained detail	No
INFO	Normal milestones	No
WARNING	Unexpected but ok	Yes
ERROR	Failure, recoverable	Yes
CRITICAL	Fatal failure	Yes

Set the level, filter the noise

`logging.basicConfig(level=logging.DEBUG)`
shows everything; `level=logging.WARNING` hides
INFO and DEBUG.

Python logging in 4 lines

```
import logging
logging.basicConfig(
    level=logging.DEBUG,
    format='%(levelname)s %(message)s')

logger = logging.getLogger(__name__)
logger.debug("loop iteration %d", i)
logger.error("failed: %s", e)
```

Rule of thumb

Ship with INFO. Turn on DEBUG only when
investigating.

Demo 1: Python logging in action

Live on Terminal — run `make setup WEEK=8` first

```
cd seminar/demo/week8/workspace/logging_demo
```

```
# run with default level (WARNING and above)
```

```
python3 app.py
```

```
# show all levels
```

```
python3 app.py --level DEBUG
```

```
# show only errors
```

```
python3 app.py --level ERROR
```

```
# inspect what each level prints
```

```
cat app.py
```

You add `logger.debug("value = %d", x)` but see nothing in the output. Most likely cause:

- A `debug()` is not a valid logging method.
- B The logging level is set to `WARNING` or higher.
- C The variable `x` is `None`.
- D You need to import `logging.debug` separately.

Concept 4: Static checks catch bugs before runtime

What static analysis does

Analyses source code *without running it* to find:

- Unused imports and variables
- Type mismatches and unreachable code
- Style violations (PEP 8 — Python's official style guide)
- Security-sensitive patterns

Think of it as a second pair of eyes that never gets tired

Tools by ecosystem

pylint Python: style + errors

flake8 Python: lightweight PEP 8

mypy Python: static type checking

shellcheck Bash/sh scripts

ESLint JavaScript/TypeScript

Run lint before every commit

Many teams enforce this in CI (continuous integration — automated checks on every push). Catching a bug in lint is free; catching it in production is expensive.

Demo 2: pylint and shellcheck

```
cd seminar/demo/week8/workspace/lint_demo

cat bad_script.py           # review the code first
pylint bad_script.py        # full report with codes
flake8 bad_script.py        # PEP 8 focused
mypy bad_script.py          # type errors

cat bad_script.sh
shellcheck bad_script.sh    # quoting and logic bugs
```

pylint score out of 10 — and what comes next

Aim for 8+. A score of 3 tells you something real is wrong. Fix one issue, re-run to confirm it clears. Lint catches *static* issues; the rest of our workflow handles *runtime* bugs.

Which command checks a Bash script for quoting errors, undefined variables, and common pitfalls?

Concept 5: Reproduce before you try to fix

Reproduce first

A bug you cannot reproduce reliably is a bug you cannot fix reliably.

- Write the exact steps to trigger it.
- Identify the inputs, environment, and state.

Then isolate with binary search

Narrow down *where* the bug lives:

- Comment out half the code — still broken?
- Which function is the last with correct data?
- Add a print at the midpoint and check.

Binary search on a call chain

A → B → C → D → crash

Check output of B: OK?

↓ yes

Check output of C: wrong!

⇒ bug is **inside C**

Key rule

Intermittent bug? Look for timing, ordering, or concurrency issues.

Your program crashes inconsistently. Which approach helps you reproduce it reliably?

- A Run it many times until you see the pattern in the output.
- B Record exact inputs, environment, and state when the crash occurs.
- C Rewrite the function from scratch — it is probably just bad code.
- D Ask a colleague to try it on their machine.

Concept 6: Debuggers let you stop time

What a debugger gives you

- **Breakpoint**: pause execution at a chosen line.
- **Step**: execute one line at a time.
- **Inspect**: read any variable's value mid-run.
- **Call stack**: see the full chain of function calls.
- **Watch**: monitor an expression across steps.

Why not just use print?

print debugging does not scale: you cannot inspect arbitrary state, pause mid-loop, or navigate the call chain.

Which debugger to use

- **GDB** C/C++ on Linux — the standard for systems code. Covered today.
- **LLDB** C/C++ on **macOS only** — same commands as GDB with slightly different syntax.
- **VS Code** GUI debugger for Python, JS, Go, C and more — the one you will use most days.

Concept 7: GDB and LLDB inspect native programs

Step 0: compile with -g (debug symbols)

```
gcc -g -o crash crash.c # embeds line numbers
```

Essential commands

```
gdb ./crash      # start GDB
(gdb) break main # breakpoint at main
(gdb) run        # run the program
(gdb) next       # step over
(gdb) step       # step into function
(gdb) print total # inspect variable
(gdb) backtrace  # show call stack
(gdb) quit
```

LLDB equivalents (macOS)

```
lldb ./crash
(lldb) b main
(lldb) run
(lldb) next
(lldb) step
(lldb) p total
(lldb) bt
(lldb) continue
(lldb) quit
```

First command after a crash

Run backtrace (bt) — it shows exactly where you were and how you got there.

Demo 3: GDB on a crashing C program

Live on Terminal

```
cd seminar/demo/week8/workspace/gdb_demo

gcc -g -o crash crash.c # compile with debug symbols
./crash                 # crashes with signal

gdb ./crash
(gdb) run               # reproduce the crash inside GDB
(gdb) backtrace         # where did we crash?
(gdb) frame 1           # move to the calling frame
(gdb) print i           # inspect loop variable
(gdb) print n           # inspect bound
(gdb) list              # show source around current line
(gdb) quit
```

After a crash inside GDB, which command shows you the chain of function calls that led to the crash?

- A print stack
- B backtrace
- C list
- D info frames

Concept 8: VS Code shows state while code runs

Key UI elements

- Breakpoint** Red dot in the gutter. Click to set/unset.
- Variables** All locals and globals at the current pause.
- Watch** Evaluate any expression at each step.
- Call Stack** Full call chain — click any frame to inspect locals.

Debug toolbar shortcuts

F5	Continue (run to next bp)
F10	Step Over
F11	Step Into
Shift+F5	Stop

launch.json snippet

```
{
  "configurations": [{
    "name": "Debug Python",
    "type": "debugpy",
    "request": "launch",
    "program": "${file}"
  }]
}
```

Demo 4: VS Code debugger on a Python script

Live in VS Code — open workspace/vscode_demo/buggy.py

```
python3 buggy.py          # output is wrong -- no crash yet
```

- # 1. Set a breakpoint on the line inside the for-loop
(the line that reads: total = ...)
- # 2. Press F5 -- execution pauses at the breakpoint
- # 3. Variables panel: read the value of 'total'
- # 4. F10 (Step Over) -- watch 'total' each iteration
does it accumulate, or does it reset?
- # 5. Find the bug; fix it; F5 to verify

What is one type of bug that a debugger catches easily that print statements would miss or make harder to find?

Concept 9: Small examples make bugs easier to see

What is an MRE?

The **smallest possible program** that still triggers the bug. Remove everything that is not required for it to appear.

Why bother?

- Faster to understand: 10 lines beats 500.
- Reveals the true cause: irrelevant code hides it.
- Required to file a good bug report.
- Often, making the MRE *is* the fix — you realise the assumption was wrong.

How to make one manually

- 1 Copy the failing code into a new file.
- 2 Delete functions unrelated to the bug.
- 3 Simplify inputs (use a literal list instead of a file).
- 4 Repeat until removing one more line makes it stop failing.

Concept 10: Perses reduces code while keeping the bug

What Perses does

You give it a **program** (-input-file) and a **test script** (-test-script) that returns 0 if the bug is still present. Perses removes code while the test keeps passing.

Basic usage

```
# Perses calls: bash test.sh <candidate-file>
python3 "$1" 2>&1 | grep -q "IndexError"
java -jar perses_deploy.jar \
  --test-script test.sh \
  --input-file big_bug.py
```

The test script contract

- Exit 0 = bug still present (keep reducing)
- Exit non-zero = bug gone (stop this path)

This is the *opposite* of normal shell conventions — be careful.

Supports 20+ languages

C, C++, Python, Java, Go, Rust, JavaScript, and more. Perses uses the language's grammar — reduced programs are always syntactically valid.

Demo 5: Perses in action

```
cd seminar/demo/week8/workspace/perses_demo

cat big_bug.py      # ~50 lines; hard to spot the bug
cat test.sh         # returns 0 when IndexError appears

bash test.sh big_bug.py && echo "bug confirmed"

java -jar perses_deploy.jar \
  --test-script test.sh --input-file big_bug.py

cat perses_result/big_bug.py  # now just a few lines
```

Expected result

Perses shrinks the 50-line program to 3–5 lines: only the line that triggers `IndexError` survives.

Your Perses test script should exit with code 0 when:

- A The candidate program compiles without errors.
- B The bug is **still present** in the candidate program.
- C The candidate program runs without crashing.
- D The candidate program is shorter than the original.

Activity: apply the full workflow

Solo or in pairs — 12 minutes

workspace/activity/broken.py has three layered bugs. Apply the full debugging workflow.

Workflow checklist

- 1 **Lint:** run `pylint broken.py` — what does it flag?
- 2 **Run:** execute the script; read the traceback from the bottom up.
- 3 **Log:** the script uses logging — re-run with `-level DEBUG` for more detail.
- 4 **Isolate:** which function is the last one that receives correct data?
- 5 **Inspect:** open in VS Code and set a breakpoint at the suspicious line.
- 6 **Fix & verify:** make the change; confirm all output is correct.

Hints

Three bugs: lint issue, logic bug, crash — fix in that order. Correct output: `Total: 42` with no errors.

Which step in the debugging workflow — reproduce, isolate, inspect, fix, or verify — do you find hardest, and why?

What should feel different after today?

You should now be able to

- Apply reproduce → isolate → inspect → fix → verify.
- Read a Python traceback and name the root cause.
- Choose a log level that matches the severity of an event.
- Run `pylint` or `shellcheck` before committing.
- Set a breakpoint in GDB or VS Code and inspect variables.
- Write a Perses test script and reduce a buggy program.

The tools at a glance

When	Reach for
Before running	Lint (pylint, shellcheck)
Reading output	Log levels
C/C++ crash	GDB / LLDB
Python/JS logic	VS Code debugger
Reporting a bug	MRE + Perses

Rank these debugging approaches from **first** to **last** in your typical debugging order:

- Run a linter on the source
- Read the error message / traceback
- Add logging and re-run
- Set a breakpoint in the debugger

Answer individually

- 1 A Python program crashes with `KeyError: 'email'`. Which file and line would you check first, and how?
- 2 You have a 400-line Python script that triggers a `ZeroDivisionError`. Describe how you would write the Perses test script and what you would expect the reduced output to look like.
- 3 *Reflection*: Describe a bug you have encountered (or imagine one) where applying reproduce → isolate → inspect would have saved time compared to random guessing.

Name one debugging tool or technique from today that you will try on your next bug.

Coming up

- **Applied class:** you will work through a multi-bug program using the full workflow.
- **Week 9:** Profiling & Performance — once you fix the bug, is the program fast enough?

Resources

- github.com/uw-pluverse/perses — download the JAR from the releases page.
- `man gdb` and `gdb -help` — built-in reference.
- code.visualstudio.com/docs/editor/debugging — VS Code debugger guide.

Questions?